

CC3092 - Deep Learning y Sistemas Inteligentes

Laboratorio #2

Redes Neuronales Convolucionales

Instrucciones generales

- Individual.
- Entrega: domingo 9 de agosto, 2026. 23:59.

1. Dataset

Trabjarán con el dataset público MNIST, un problema clásico de clasificación multiclase donde se predice el dígito (0-9) representado en una imagen de un número escrito a mano.

Dataset en torchvision: *from torchvision.datasets import MNIST*

2. Exploración y preparación de los datos

Cargue el dataset y responda las siguientes preguntas:

- ¿Cuántas observaciones (imágenes) y cuántas clases tiene el dataset? ¿Las clases están balanceadas?
- ¿Cuál es la dimensión de cada imagen y qué rango de valores tienen los píxeles?
- ¿Es necesario normalizar los valores de los píxeles?
- Visualice al menos 10 ejemplos del dataset con su etiqueta correspondiente.

Divida el conjunto de datos en entrenamiento y validación.

3. Investigación: capas de PyTorch para la CNN

Investigue las capas adicionales de torch.nn necesarias para construir una red convolucional. Para cada capa, describa brevemente su propósito y sus parámetros más relevantes. Como mínimo deben investigar:

- nn.Conv2d
- nn.MaxPool2d
- nn.AvgPool2d
- nn.BatchNorm2d
- nn.Flatten
- nn.CrossEntropyLoss

Adicionalmente, investigue brevemente el concepto de tensor en pytorch, campo receptivo (receptive field) y por qué las CNN suelen requerir menos parámetros que un MLP equivalente para procesar imágenes.

4. Construcción y entrenamiento de las arquitecturas

Construya y entrene dos arquitecturas distintas para clasificar los dígitos de MNIST:

- Un MLP (semejante al Lab #1), que reciba la imagen aplanada como vector de entrada.
- Una CNN (red neuronal convolucional), que reciba la imagen como tensor 2D y utilice al menos dos capas convolucionales.

Itere sobre cada arquitectura para obtener la mejor red posible. Se recomienda seguir una estrategia de búsqueda de hiperparámetros para explorar el espacio de forma sistemática, cambiando idealmente una o dos variables a la vez para poder atribuir el efecto de cada cambio.

Para cada iteración, registre:

- La configuración de hiperparámetros usada.
- La pérdida (loss) de entrenamiento y de validación por epoch.
- Las métricas de evaluación del problema de clasificación: accuracy, precision, recall y F1-score (macro o weighted), calculadas sobre el conjunto de validación.
- El número total de parámetros entrenables del modelo.

Grafiquen las curvas de pérdida de entrenamiento y validación de al menos 3 iteraciones por arquitectura, para poder identificar visualmente señales de overfitting o underfitting.

Una vez identificada la mejor configuración de cada arquitectura según sus métricas de validación, evalúe ambos modelos finales una única vez sobre el conjunto de test, reporte sus métricas y genere la matriz de confusión de cada uno.

5. Comparación de arquitecturas

Con los resultados de la mejor configuración de cada arquitectura, realicen una comparación directa entre el MLP y la CNN, considerando:

- Cantidad de parámetros entrenables de cada modelo.
- Desempeño (accuracy, precision, recall, F1-score) sobre el conjunto de test.
- Relación entre la cantidad de parámetros y la calidad de los resultados obtenidos.
- Tiempo de entrenamiento aproximado de cada arquitectura.

Se recomienda presentar esta comparación en una gráfica o tabla que relacione el número de parámetros de cada modelo con su accuracy en el conjunto de test.

6. Discusión y análisis

Responda con base en sus resultados:

- ¿Qué cambio de hiperparámetro tuvo el mayor impacto positivo en las métricas de validación de cada arquitectura? ¿Y el mayor impacto negativo?
- ¿Observaron overfitting o underfitting en alguna de sus iteraciones? ¿Cómo lo identificaron y qué hicieron para mitigarlo?
- ¿La regularización mejoró el desempeño en validación? ¿Cuál método funcionó mejor para cada arquitectura y por qué creen que fue así?
- Comparando el MLP y la CNN, ¿cuál obtuvo mejor desempeño en test? ¿Cómo se relaciona esa diferencia con la cantidad de parámetros de cada modelo y con la forma en que cada arquitectura procesa la información espacial de la imagen?
- ¿En qué tipo de errores se equivocan más el MLP y la CNN? Analicen las matrices de confusión de ambos modelos.
- Si tuvieran que desplegar un modelo de producción para este problema, priorizando tanto exactitud como eficiencia (número de parámetros y tiempo de inferencia), ¿qué arquitectura elegirían y por qué?

Entregables

Entregable	Contenido
PDF (máx. 3 páginas)	Investigación de las capas de PyTorch para CNN, tabla de resultados de las iteraciones (MLP y CNN), comparación de arquitecturas, análisis de resultados y conclusiones.
Repositorio (Git)	Jupyter Notebook completo y comentado: carga y preparación de datos, investigación de capas, definición de ambos modelos (MLP y CNN), entrenamiento de las 10+ iteraciones y evaluación final sobre test.

Incluyan el enlace al repositorio al final del PDF.